

本小节内容

快速排序实战

快速排序实战

上一小节我们讲解了快速排序的原理，下面我们来代码实战。

代码实战步骤：首先我们通过随机数生成 10 个元素，通过随机数生成，我们可以多次测试排序算法是否正确，然后打印随机生成后的元素顺序，然后通过快速排序对元素进行排序，然后再次打印排序后的元素顺序。

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

typedef int ElemType;
typedef struct{
    ElemType *elem;//存储元素的起始地址
    int TableLen;//元素个数
}SSTable;

void ST_Init(SSTable &ST,int len)
{
    ST.TableLen=len;
    ST.elem=(ElemType *)malloc(sizeof(ElemType)*ST.TableLen);//申请一块堆空间，当数组来使用
    int i;
    srand(time(NULL));//随机数生成，每一次执行代码就会得到随机的 10 个元素
    for(i=0;i<ST.TableLen;i++)
    {
        ST.elem[i]=rand()%100;//生成的是 0-99 之间
    }
}

//打印数组中的元素
void ST_print(SSTable ST)
{
    for(int i=0;i<ST.TableLen;i++)
    {
        printf("%3d",ST.elem[i]);
    }
    printf("\n");
}
```

//交换两个元素

```
void swap(ElementType &a, ElementType &b)
{
    ElementType tmp;
    tmp=a;
    a=b;
    b=tmp;
}
```

// 64 94 95 79 69 84 18 22 12 78

//比 64 小的放在左边，比 64 大的放在右边

```
int Partition(ElementType A[], int low, int high)
{
    ElementType pivot=A[low]; //首先使用左边变量存储分割值
    while(low<high)
    {
        while(low<high&&A[high]>=pivot) //从后往前遍历，找到一个比分割值小的
            --high;
        A[low]=A[high]; //把比分割值小的那个元素，放到 A[low]
        while(low<high&&A[low]<=pivot) //从前往后遍历，找到一个比分割值大的
            ++low;
        A[high]=A[low]; //把比分割值大的那个元素，放到 A[high], 因为刚才 high 位置的元素已经放到了 low 位置
    }
    A[low]=pivot;
    return low; //返回分隔值所在的下标
}
```

//递归实现

```
void QuickSort(ElementType A[], int low, int high)
{
    if(low<high)
    {
        int pivotpos=Partition(A, low, high); //分割点左边的元素都比分割点要小，右边的比分割点大
        QuickSort(A, low, pivotpos-1);
        QuickSort(A, pivotpos+1, high);
    }
}
```

//《王道 C 督学营》课程

//快速排序

```
int main()
{
    SSTable ST;
    ST_Init(ST, 10); //初始化
}
```

```
ElemType A[10]={ 64, 94, 95, 79, 69, 84, 18, 22, 12, 78};  
//内存 copy 接口, 当你 copy 整型数组, 或者浮点型时, 要用 memcpy, 不能用 strcpy, 初试考 memcpy  
概率很低  
memcpy(ST.elem, A, sizeof(A)); //这是为了降低调试难度, 每次数组数据固定而设计的  
ST_print(ST);  
QuickSort(ST.elem, 0, 9); //注意这个位置是 n-1, 也就是 9, 因为函数里取了 high 位置的值  
ST_print(ST);  
return 0;  
}
```

假如每次快速排序数组都被平均地一分为二, 那么可以得出 QuickSort 递归的次数是 $\log_2 n$, 第一次 partition 遍历次数为 n , 分成两个数组后, 每个数组遍历 $n/2$ 次, 加起来还是 n , 因此时间复杂度是 $O(n \log_2 n)$, 因为计算机是二进制的, 所以在复试面试回答复杂度或与人交流时, 提到复杂度时一般直接讲 $O(n \log n)$, 而不带下标 2。快速排序最差的时间复杂度为什么是 n^2 呢? 因为数组本身从小到大有序时, 如果每次我们仍然用最左边的数作为分割值, 那么每次数组都不会二分, 导致递归 n 次, 所以快速排序最坏时间复杂度为 n 的平方。当然, 为了避免这种情况, 有时会首先随机选择一个下标, 先将对应下标的值与最左边的元素交换, 再进行 partition 操作, 从而极大地降低出现最坏时间复杂度的概率, 但是仍然不能完全避免。

因此快排最好和平均时间复杂度是 $O(n \log_2 n)$, 最差是 $O(n^2)$ 。

快排的空间复杂度是 $O(\log_2 n)$, 因为递归的次数是 $\log_2 n$, 而每次递归的形参都是需要占用空间的。